

Real-Time Collaborative Code Editor

Software Engineering Project Proposal

Submitted by: Yuvraj Jasuja (1024030069) and Harwinder (1024030075)

Department of Computer Engineering

Thapar Institute of Engineering and Technology

August 2026

1 Higher-order Goal

The goal of this project is to make collaborative programming more accessible and interactive by providing a shared environment where multiple users can work on the same code in real time.

Programming is often a collaborative activity. Students work together on assignments, developers pair program while solving problems, and teams frequently need to discuss and modify the same codebase. However, basic collaboration often involves repeatedly sharing files, copying code through messaging platforms, or manually merging different versions of the same program.

Our project aims to reduce this friction by creating a real-time coding environment in which users can join a common room and collaborate on the same code simultaneously.

2 Introduction / Problem Statement

Collaborative coding can become inefficient when multiple people are working on the same programming task.

Consider a common classroom situation where two students are solving a programming problem together. One student writes a part of the solution and sends the file to the other. The second student modifies it and sends another version back. As the number of changes increases, it becomes difficult to keep track of the latest version of the program.

This workflow creates several problems:

- Repeated sharing of source code files.
- Multiple versions of the same program.
- Difficulty identifying the latest version.
- Delayed feedback between collaborators.
- Version conflicts when different users modify the same code.
- Lack of a simple shared environment for collaborative programming.

The Real-Time Collaborative Code Editor addresses this problem by allowing multiple users to join a shared coding room and view code changes as they happen.

3 Objectives

- Build a web-based platform for real-time collaborative code editing.
- Allow users to create or join coding rooms using a unique Room ID.
- Synchronize code changes between multiple connected users in real time.
- Display active users currently connected to a coding room.
- Provide a clean and interactive code editing environment.
- Ensure that users joining an existing room can receive the current state of the code.
- Extend the platform in future phases to support multiple programming languages and secure code execution.

4 Methodology / Solution Approach

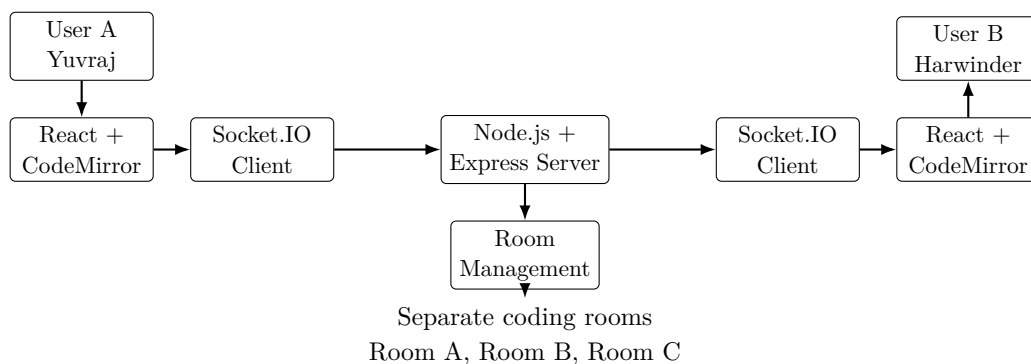
4.1 System Architecture

The system follows a client-server architecture with real-time communication between users and the backend server.

The main components are:

- React-based user interface.
- CodeMirror-based code editor.
- Socket.IO for real-time communication.
- Node.js and Express backend server.
- Room-based communication for separate collaborative sessions.

4.2 System Architecture Flow Diagram



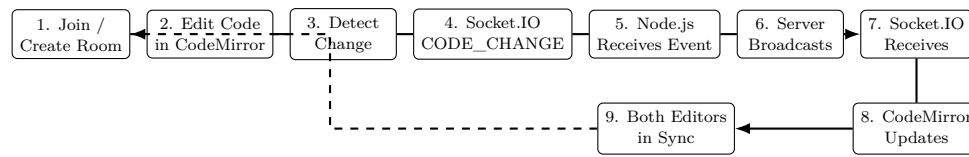
The browser contains the React interface and CodeMirror editor. Socket.IO maintains the real-time connection with the Node.js and Express server. The server manages room membership and broadcasts changes to users in the same room.

4.3 Real-Time Collaboration Flow

The main workflow is:

1. A user enters a username and creates a new coding room or enters an existing Room ID.
2. The user establishes a Socket.IO connection with the backend server.
3. The server associates the user with the requested coding room.
4. Other users join the same room using the Room ID.
5. When a user modifies the code, CodeMirror detects the change.
6. The updated code is emitted to the backend using Socket.IO.
7. The server broadcasts the update to other users in the same room.
8. Connected users receive the update and their editor reflects the new code.
9. When a user disconnects, the server notifies the remaining users.

4.4 Real-Time Code Synchronization Flow Diagram



This diagram shows the core operation: a change made by one user is detected, transmitted through Socket.IO, processed by the Node.js server, broadcast to room members, and reflected in the other editor.

4.5 Room-Based Communication

Each collaborative session is organized as a separate room. This prevents code changes from one group of users from being sent to unrelated users.

For example:

- Room A may contain Yuvraj and Harwinder.
- Room B may contain other users.

Code changes generated inside Room A are only broadcast to users connected to Room A.

4.6 Future Multi-Language Execution

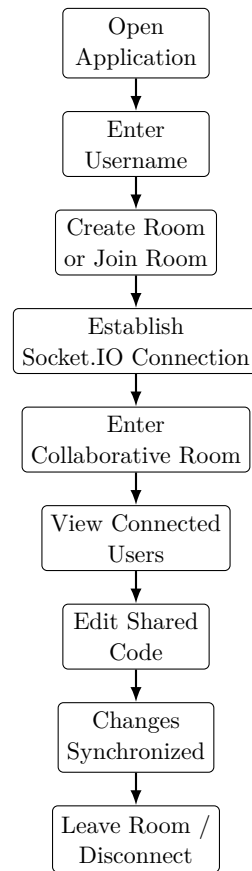
The current version focuses primarily on real-time collaborative editing. A future extension will allow users to select languages such as C++, Java, Python, JavaScript, and C.

Users will be able to write code, provide input, execute the program, and view output or compilation errors. Because arbitrary code should not be executed directly on the main application server, execution will use a secure sandbox or external execution service.

Code Editor → Secure Sandbox / Execution API → Compiler / Runtime → Output

4.7 User Journey Flow Diagram

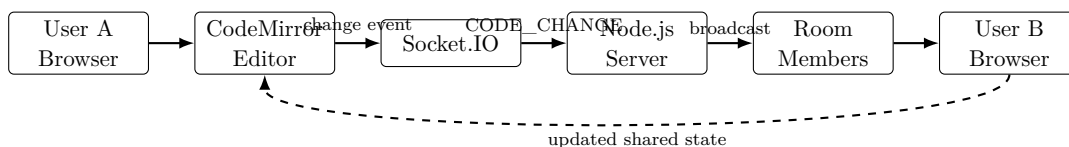
The following flow represents the complete journey of a user from opening the application to collaborating inside a coding room.



This flow gives a simple view of the user's interaction with the system. The user first enters the application, joins a room, connects to the real-time server, and then collaborates through the shared editor.

4.8 Client-Server Communication Flow

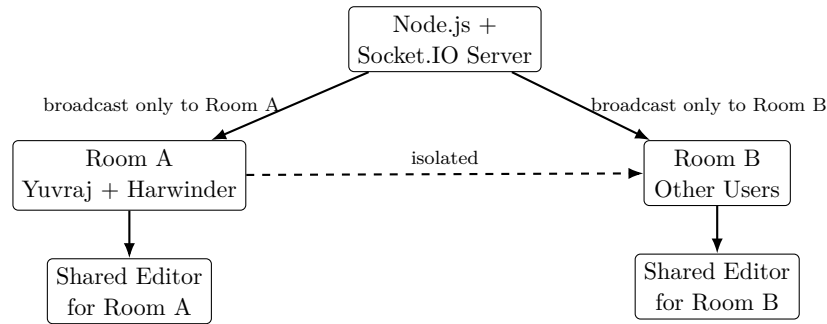
The project uses event-based communication between the browser and the backend server. The simplified communication sequence is shown below.



The important point is that the server does not need to continuously refresh the browser. Instead, the Socket.IO connection allows events to be sent as soon as a change occurs.

4.9 Room Isolation Flow Diagram

Room isolation ensures that a collaboration session remains separate from other sessions.



A change generated in Room A is distributed only to the users belonging to Room A. Similarly, Room B remains independent. This room-based design allows several coding sessions to operate at the same time.

4.10 Future Code Execution Flow Diagram

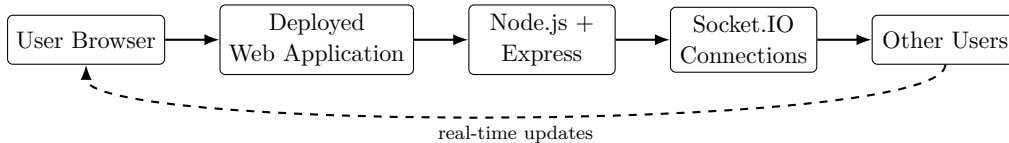
The collaborative editor can later be extended into a multi-language online coding platform. The planned execution path is shown below.



This is a planned extension rather than a claim about the current implementation. The execution component will be isolated from the main application server so that arbitrary user programs are not executed directly on the collaboration server.

4.11 Deployment Flow Diagram

The application can be deployed as a web service so that collaborators can access the same coding room from different systems.



Deployment allows the same collaborative environment to be accessed over the internet rather than only from a local development machine.

5 Tech Stack

- **React:** Frontend user interface.
- **CodeMirror:** Interactive code editing environment.
- **Socket.IO:** Real-time, bidirectional communication.
- **Node.js:** Backend runtime.
- **Express.js:** Backend server and routes.
- **UUID:** Unique Room ID generation.
- **Render:** Possible deployment platform.

6 Evaluation Criteria

- **Synchronization Accuracy:** Code changes are correctly reflected for users in the same room.
- **Real-Time Response:** Low delay between editing and receiving updates.
- **Room Isolation:** Users in different rooms do not receive unrelated updates.
- **Connection Handling:** Correct join and disconnect behavior.
- **Concurrent Users:** Ability to support multiple users in a shared room.
- **Usability:** Users can easily create, join, and use a room.

7 Scalability

The architecture is based on independent real-time rooms. Potential improvements include multiple backend instances, a distributed Socket.IO adapter, persistent project storage, database support, and a separate code execution service.

8 Resources / Tooling

- React and JavaScript for frontend development.
- CodeMirror for the coding interface.
- Node.js and Express for backend development.
- Socket.IO for real-time communication.
- GitHub for version control and project management.
- Render or another cloud platform for deployment.
- Secure execution API or sandbox for future code execution.

9 Project Scope and Deliverables

9.1 Initial Deliverable

- Home page for username and Room ID.
- Create a unique coding room.
- Join an existing coding room.
- Real-time collaborative code editing.
- Connected-user display.
- Room-based Socket.IO communication.
- User connection and disconnection handling.

9.2 Subsequent Deliverables

- Multiple programming language support.
- Language-based syntax highlighting.
- Run code functionality.
- Custom input and output terminal.
- Compilation and runtime error display.
- Execution time information.
- User authentication.
- Saving and loading projects.
- Multi-file project support.
- Live cursor tracking.
- Integrated collaborative chat.

10 Risks and Mitigations

- **Synchronization conflicts:** Future versions can use operational transformation or CRDT-based collaboration for more advanced concurrent editing.
- **Network latency:** Socket.IO reconnection support can improve reliability.
- **User disconnection:** Disconnect events update the active-user list.
- **Unauthorized room access:** Future authentication and private-room controls can be added.
- **Code execution security:** Future execution will use an isolated sandbox or execution service.

11 Timeline

Phase	Focus
Weeks 1–2	Requirement analysis, project setup, React frontend, and room creation interface.
Weeks 3–4	CodeMirror integration and basic code editing.
Weeks 5–6	Socket.IO integration and real-time synchronization.
Weeks 7–8	Room management, connected users, and join/disconnect handling.
Weeks 9–10	Multi-user testing, UI improvements, and deployment.
Weeks 11–12	Prototype of language selection and secure code execution integration.

12 Summary

The Real-Time Collaborative Code Editor is designed to simplify collaborative programming by providing a shared environment where multiple users can work on the same code simultaneously.

The platform combines React and CodeMirror for the frontend coding experience with Node.js, Express,

and Socket.IO for real-time communication. Users can create or join coding rooms, collaborate with other users, and see code changes as they happen.

The current project focuses on real-time code synchronization. Future development will extend the platform with multi-language support, secure code execution, project storage, authentication, and additional collaborative features.

Ultimately, the project aims to evolve from a basic real-time editor into a practical collaborative coding environment for students, developers, and teams.